

UNIVERSITETI I EVROPËS JUGLINDORE
УНИВЕРЗИТЕТ НА ЈУГОИСТОЧНА ЕВРОПА
SOUTH EAST EUROPEAN UNIVERSITY

FACULTY OF CONTEMPORARY SCIENCES AND TECHNOLOGIES
UNDERGRADUATE STUDIES IN THE FIELD OF COMPUTER SCIENCE

MapOut

Week 5: Informed Search and Heuristics

Student: Mirjana Arsenoska
Student ID: 131834

Professor: Prof. Dr. Azir Aliu
Course: Introduction to Artificial Intelligence

Tetovo, 2026

CONTENT

Introduction 1
1. Informed Search and Heuristics 2
1.1 Greedy Best-First Search 2
1.2 A* Search 3
1.3 Comparison of Search Algorithms 3
2. Applying Informed Search to MapOut 4
3. Dataset Expansion 5
4. Baseline Implementation 6
4.1 Baseline Testing 6
4.2 Current Limitations 8
Conclusion 9
AI Assistance Disclosure 9

Introduction

In Week 4, MapOut was represented as a search problem, and uninformed search algorithms such as Breadth-First Search (BFS) and Depth-First Search (DFS) were explored. These algorithms follow specific search strategies without knowing which possible state is closer to the goal.

In Week 5, the focus is on informed search algorithms, which use additional information to make better decisions during the search. The main concepts are heuristic functions, Greedy Best-First Search, and A* Search.

For MapOut, these methods are useful because planning a trip involves choosing suitable attractions while considering distance, available time, budget, and user preferences.

During this week, the attraction dataset was also expanded, and a simple baseline itinerary generator was implemented and tested.

1. Informed Search and Heuristics

Informed search is a search technique that uses additional knowledge about the problem to decide which states should be explored first. Instead of exploring possible solutions without guidance, the algorithm estimates which options are more promising and prioritizes them.

This additional knowledge is provided through a heuristic function, written as **h(n)**. A heuristic estimates the remaining cost from the current state to the goal. It does not necessarily provide the exact cost, but it helps the algorithm make more informed decisions.

For example, imagine a traveler in Ohrid who wants to reach the Church of St. John at Kaneo. MapOut could use the straight-line distance from each possible location to the church as a heuristic. A location estimated to be closer to the destination would receive a lower heuristic value and could be explored first.

The quality of the heuristic is important. A useful heuristic can reduce unnecessary searching, while an inaccurate one may guide the algorithm toward a less efficient route.

1.1 Greedy Best-First Search

Greedy Best-First Search is an informed search algorithm that always chooses the state that appears closest to the goal according to the heuristic function.

Its evaluation function is: **f(n) = h(n)**

Here, **f(n)** determines which state should be explored next, while **h(n)** estimates the remaining cost to the goal. The algorithm selects the state with the lowest heuristic value.

For example, suppose a traveler wants to reach the Church of St. John at Kaneo and has two possible locations to explore. The first is estimated to be 300 meters from the destination, while the second is estimated to be 700 meters away. Greedy Best-First Search would prioritize the first location because it has a lower heuristic value.

The main advantage of this algorithm is that it can find a solution quickly when the heuristic provides good guidance. However, it only considers the estimated remaining cost and ignores the cost already spent reaching the current location.

Because of this, Greedy Best-First Search does not guarantee the shortest or least expensive route. A location may appear closer to the goal but require a longer actual route because of roads or other obstacles.

1.2 A* Search

A* Search is an informed search algorithm that considers both the cost already spent and the estimated cost remaining to reach the goal.

Its evaluation function is: **f(n) = g(n) + h(n)**

In this formula, **g(n)** represents the actual cost from the starting state to the current state, while **h(n)** estimates the remaining cost from the current state to the goal. The algorithm selects the state with the lowest combined value.

For example, suppose MapOut is comparing two possible routes to the same destination. For the first route, the traveler has already spent 20 minutes traveling, and the estimated remaining travel time is 10 minutes. Its total estimated cost is **f(n) = 20 + 10 = 30 minutes**. For the second route, the traveler has spent 10 minutes traveling, and the estimated remaining travel time is 15 minutes. Its total estimated cost is **f(n) = 10 + 15 = 25 minutes**.

A* would prioritize the second route because its total estimated cost is lower, even though its estimated remaining travel time is greater.

The main advantage of A* is that it evaluates the overall estimated route cost instead of focusing only on the remaining distance. With an admissible heuristic, meaning one that never overestimates the remaining cost, and appropriate search conditions, A* can find an optimal solution. However, A* may require more memory and processing because it keeps track of different possible states and their costs.

1.3 Comparison of Search Algorithms

The main difference between uninformed and informed search algorithms is how they choose which state to explore next.

BFS explores states level by level, starting with the states closest to the initial state in terms of the number of actions. **DFS** explores one path as deeply as possible before returning to explore alternatives. Neither uses a heuristic to guide the search.

Greedy Best-First Search uses only the estimated remaining cost, making it useful for quickly exploring promising options. **A*** considers both the accumulated cost and the estimated remaining cost, making it more suitable for finding efficient routes.

For MapOut, BFS and DFS can explore possible attraction combinations, while Greedy Best-First Search and A* could help prioritize routes based on estimated travel distance or time.

2. Applying Informed Search to MapOut

MapOut is designed to generate personalized travel itineraries based on a user's destination, interests, budget, and available time.

As defined in Week 4, a search state contains the current location, remaining budget, remaining time, and attractions already visited. Each action represents choosing another attraction, provided that the selection satisfies the user's constraints.

For informed search, MapOut could use geographical distance or estimated travel time as a heuristic. For example, if a user wants to visit several historical attractions in Ohrid, the agent could prioritize attractions that are closer to the current location while checking whether the visit fits within the remaining time and budget.

However, the closest attraction is not always the most suitable one. An attraction may be nearby but unrelated to the user's interests, or it may have an entrance fee that exceeds the available budget. Therefore, MapOut must consider user preferences and constraints alongside the search strategy.

During Week 5, a simple rule-based baseline was implemented to generate itineraries. This provides a working starting point before introducing more advanced AI planning methods. Greedy Best-First Search and A* have been studied theoretically but have not yet been implemented.

3. Dataset Expansion

During Week 4, the initial MapOut dataset was created with 15 attractions in Ohrid and the surrounding area.

During Week 5, the dataset was expanded to **101 attractions across eight destinations** in North Macedonia: Ohrid, Skopje, Tetovo, Gostivar, Bitola, Mavrovo, Struga, and Kruševo.

The expanded dataset supports different categories, including history, culture, nature, adventure, food, relaxation, and entertainment. Each attraction can belong to multiple categories, allowing it to match more than one user interest.

The dataset contains information such as attraction name, city, region, categories, geographical coordinates, estimated cost, visit duration, availability, and source.

To organize the data processing, two Python modules were implemented:

- src/data_loader.py – loads attraction records from the CSV dataset.
- src/validator.py – checks the records for missing or invalid information.

The validation process checks attraction IDs, names, locations, categories, coordinates, costs, and visit durations, among other fields.

After validation, all 101 attractions passed the implemented checks without warnings. This confirms that the records follow the defined data structure and validation rules, although individual real-world details may still require verification.

Unknown estimated costs are kept as missing values rather than being treated as free attractions.

The expanded dataset provides a foundation for generating itineraries across multiple destinations and for implementing more advanced planning methods in future weeks.

4. Baseline Implementation

A baseline is a simple working solution used as a starting point for a project. It allows the developer to test the basic functionality before implementing more advanced algorithms.

During Week 5, a rule-based baseline itinerary generator was implemented in src/planner.py. The baseline generates itineraries using four user preferences: destination, interests, budget, and available visit time.

The algorithm examines attractions in dataset order. For each attraction, it checks whether the attraction belongs to the selected city and matches at least one of the user's preferred categories.

It then checks whether the attraction fits within the remaining budget and available visit time. Attractions with unknown estimated costs are skipped. If an attraction satisfies all conditions, it is added to the itinerary, and the total estimated cost and visit duration are updated.

The baseline does not yet calculate geographical distances, travel time between attractions, or the most efficient visiting order. Its purpose is to demonstrate that MapOut can generate basic itineraries using the dataset and user preferences.

4.1 Baseline Testing

The baseline was tested using tests/test_planner.py, which contains test scenarios.

Test	Destination	Interests	Budget	Available Time
1	Ohrid	History, Culture	€10	180 minutes
2	Gostivar	Nature, Relaxation	€5	180 minutes
3	Tetovo	History, Culture	€5	180 minutes

The tests use assertions to verify that the generated itineraries contain attractions from the correct destination, match the selected interests, and stay within the specified budget and visit-duration limits.

Test 1 – Ohrid

The baseline generated an itinerary containing Samuel's Fortress, Robevci Family House, and Bay of Bones Museum. The total estimated attraction cost was €7.50, and the total visit duration was 180 minutes.

```
===== MapOut Baseline Itinerary =====
Destination: Ohrid
- Samuel's Fortress
- Robevci Family House
- Bay of Bones Museum
Total estimated cost: 7.5 EUR
Total visit duration: 180 minutes
==================================
```

Test 2 – Gostivar

The baseline generated an itinerary containing Vardar River Source at Vrutok and Gostivar City Park / Vardar Promenade. The total estimated attraction cost was €0, and the total visit duration was 120 minutes.

```
===== MapOut Baseline Itinerary =====
Destination: Gostivar
- Vardar River Source at Vrutok
- Gostivar City Park / Vardar Promenade
Total estimated cost: 0.0 EUR
Total visit duration: 120 minutes
==================================
```

Test 3 – Tetovo

The baseline was also tested using Tetovo with History and Culture as selected interests, a budget of €5, and 180 minutes of available visit time. The total estimated attraction cost was €0, and the total visit duration was 180 minutes.

```
===== MapOut Baseline Itinerary =====
Destination: Tetovo
- Painted Mosque (Šarena Džamija)
- Arabati Baba Tekke
- Tetovo Fortress (Baltepe)
Total estimated cost: 0.0 EUR
Total visit duration: 180 minutes
==================================
```

Testing Results

The three test scenarios demonstrated that the baseline can generate itineraries for different destinations and user preferences. The automated tests checked that the selected attractions matched the requested city and interests while remaining within the specified budget and visit-duration limits.

The results confirm that the baseline follows the implemented selection rules. However, the tests do not evaluate route efficiency or travel time between attractions.

4.2 Current Limitations

Although the baseline can generate basic itineraries, it still has several limitations.

Attraction selection: The algorithm selects attractions in the order they appear in the dataset rather than searching for the best possible combination. This means that some suitable attractions may be skipped even when another combination would provide a better itinerary.

Travel time and distance: The baseline calculates only the time spent visiting attractions. It does not use geographical coordinates to calculate distances or estimate travel time between locations. Therefore, the actual trip may take longer than the calculated visit duration.

Route optimization: The selected attractions are not organized according to their geographical locations. As a result, the itinerary may require unnecessary travel between attractions.

Unknown costs: Attractions with unknown estimated costs are skipped because their affordability cannot be confirmed. This may exclude otherwise suitable attractions.

Availability and opening hours: Although the dataset contains availability information, the baseline does not yet use it to check whether attractions are open during the planned visit.

User interaction: The current implementation is tested using predefined preferences. Users cannot yet enter their own destination, interests, budget, and available time through an interactive interface.

These limitations will be considered in future development, particularly through constraint satisfaction, planning, and improved route calculations.

Conclusion

During Week 5, the theoretical concepts of informed search and heuristic functions were studied, including Greedy Best-First Search and A*. Examples were used to explain how these algorithms could later be applied to MapOut.

The practical work included expanding the dataset to 101 attractions across eight destinations, implementing data loading and validation, and developing a working baseline itinerary generator.

The baseline was tested using Ohrid, Gostivar, and Tetovo scenarios to check whether the generated itineraries satisfy the selected constraints.

The next stage of MapOut will focus on constraint satisfaction and planning. This will allow the project to explore more advanced methods for selecting attractions while considering user preferences, budget, and time limitations.

AI Assistance Disclosure

ChatGPT (OpenAI) was used to explain informed search concepts, provide examples of Greedy Best-First Search and A*, assist with drafting and organizing the theoretical report, and support the development and testing of the Python baseline planner.

AI assistance was also used to discuss dataset expansion, validation, code organization, and test scenarios. The implementation was executed and tested in Visual Studio Code, and the generated outputs were reviewed as part of the project work.